

Технічний пакет: автоматизація відділу продажу (Мілена → Міленіум)

Мета документа — дати Греку все необхідне, щоб почати програмування бота продажу без додаткових уточнень: структуру БД, приклади даних, логіку діалогу і правила безпеки.

1. Архітектура: 5 + 3 таблиці

Базові 5 (з попереднього документа) + 3 додаткові, потрібні для реальної CRM-роботи, а не лише FAQ:

Таблиця	Призначення
<code>services</code>	Каталог послуг і підпослуг
<code>intents</code>	Що може запитати клієнт (тригери)
<code>knowledge_cards</code>	Самі відповіді у структурованому вигляді
<code>dialog_flows</code>	Послідовність питань і переходів
<code>handoff_rules</code>	Коли бот передає людину спеціалісту
<code>conversations</code>	Сесія діалогу з конкретним клієнтом (додано)
<code>messages</code>	Історія повідомлень усередині conversation (додано)
<code>clients</code>	Картка клієнта після Етапу 8 (додано)

Без останніх трьох Грек не зможе підключити стан діалогу до реальної CRM (Supabase) — вони лежать в основі "пам'яті" бота між повідомленнями.

2. SQL-схема (Postgres / Supabase)

-- 2.1 Каталог послуг

```

create table services (
  id uuid primary key default gen_random_uuid(),
  direction text not null,          -- напр. 'Нотаріус', 'Карта побиту', 'Сімейні
  subservice text,                  -- напр. 'Довіреність на нерухомість'
  status text not null default 'needs_legal_review'
  check (status in ('actual','needs_update','historical','do_not_use','needs_le
  created_at timestampz default now(),
  updated_at timestampz default now()
);

```

-- 2.2 Наміри клієнта (тригери)

```

create table intents (
  id uuid primary key default gen_random_uuid(),
  service_id uuid references services(id) on delete cascade,
  stage text not null,              -- 'Перше звернення','Кваліфікація','Докумен
  intent_label text not null,        -- 'Питає про вартість'
  trigger_phrases text[] not null,   -- масив тригер-фраз для матчингу/embedding
  created_at timestampz default now()
);

```

-- 2.3 Картки знань (основний контент)

```

create table knowledge_cards (
  id text primary key,              -- напр. 'notary_real_estate_documents_001'
  service_id uuid references services(id) on delete cascade,
  stage text not null,
  intent text not null,
  required_context text[],
  answer_short text not null,
  answer_full text not null,
  disclaimer text,
  next_question text,
  next_action text,                 -- 'collect_documents','offer_consultation',
  handoff_condition text[],
  status text not null default 'needs_legal_review'
  check (status in ('actual','needs_update','historical','do_not_use','needs_le
  last_reviewed_at timestampz,      -- ключове поле, якого бракувало
  source text,
  created_at timestampz default now(),
  updated_at timestampz default now()
);

```

-- 2.4 Логіка діалогу (стейт-машина)

```

create table dialog_flows (
  id uuid primary key default gen_random_uuid(),
  service_id uuid references services(id) on delete cascade,
  current_stage text not null,
  required_fields text[] not null,   -- обов'язкові для переходу далі

```

```

    optional_fields text[],                -- можна пропустити
    next_stage_on_complete text,
    next_stage_on_incomplete text,        -- напр. повторне уточнення
    max_clarifying_questions int default 2, -- анти-переспам
    created_at timestamptz default now()
);

-- 2.5 Правила передачі людині
create table handoff_rules (
    id uuid primary key default gen_random_uuid(),
    service_id uuid references services(id),
    condition_label text not null,        -- 'спадкова справа', 'спір між сторонами'
    handoff_to text not null,            -- 'Мілена', 'Олександр', 'юрист', 'адвокат'
    priority text default 'normal' check (priority in ('normal', 'urgent')),
    created_at timestamptz default now()
);

-- 2.6 Клієнти
create table clients (
    id uuid primary key default gen_random_uuid(),
    full_name_latin text,
    birth_date date,
    parents_names text,
    pesel text,
    address text,
    phone_pl text,
    email text,
    preferred_contact text,
    case_summary text,
    responsible_person text,
    created_at timestamptz default now()
);

-- 2.7 Розмови (сесії)
create table conversations (
    id uuid primary key default gen_random_uuid(),
    client_id uuid references clients(id),
    channel text not null,                -- 'facebook', 'tiktok', 'viber', 'whatsapp'
    service_id uuid references services(id),
    current_stage text,
    collected_context jsonb default '{}'::jsonb, -- накопичені відповіді клієнта
    status text default 'active' check (status in ('active', 'handed_off', 'closed'))
    created_at timestamptz default now(),
    updated_at timestamptz default now()
);

-- 2.8 Повідомлення

```

```

create table messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid references conversations(id) on delete cascade,
  sender text not null check (sender in ('client','bot','human_agent')),
  content text not null,
  knowledge_card_id text references knowledge_cards(id),
  created_at timestamptz default now()
);

```

Індекси, які варто одразу додати:

```

create index idx_knowledge_cards_service on knowledge_cards(service_id);
create index idx_knowledge_cards_status on knowledge_cards(status);
create index idx_conversations_client on conversations(client_id);
create index idx_messages_conversation on messages(conversation_id);

```

3. Приклади готових knowledge_cards (JSON для сідування БД)

```

[
  {
    "id": "residence_card_price_001",
    "service": "Карта побиту",
    "stage": "Вартість",
    "intent": "Скільки коштує оформлення карти побиту",
    "required_context": ["підстава подачі", "воєводський ужонд", "чи вже подано д",
    "answer_short": "Вартість нашої роботи залежить від підстави подачі та складн",
    "answer_full": "Вартість супроводу формується індивідуально після аналізу під",
    "disclaimer": "Фінальна ціна фіксується після безкоштовної консультації та ан",
    "next_question": "На якій підставі плануєте подавати документи, і чи вже було",
    "next_action": "offer_consultation",
    "handoff_condition": ["відмова в минулому", "судове оскарження"],
    "status": "needs_update",
    "last_reviewed_at": null,
    "source": "Milena chat history"
  },
  {
    "id": "marriage_foreigner_court_permit_001",
    "service": "Сімейні справи",
    "stage": "Пояснення послуги",
    "intent": "Одруження з іноземцем у Польщі без документа про можливість шлюбу",
    "required_context": ["громадянство сторін", "попередні шлюби", "наявність доз

```

```

    "answer_short": "Якщо немає документа про можливість укладення шлюбу за закон
    "answer_full": "Процедура складається з подання заяви до суду, отримання дозв
    "disclaimer": "Точний перелік документів і строки уточнюються після аналізу г
    "next_question": "Яке громадянство має кожна сторона, і чи були попередні шлю
    "next_action": "collect_documents",
    "handoff_condition": ["один з подружжя неповнолітній", "попередній шлюб не ро
    "status": "needs_legal_review",
    "last_reviewed_at": null,
    "source": "Milena chat history"
  }
]

```

4. Логіка діалогу (псевдокод стейт-машини)

```

FUNCTION handle_incoming_message(conversation, message):

    intent = classify_intent(message.text, conversation.service_id)

    IF intent.confidence < THRESHOLD:
        RETURN ask_clarifying_question(conversation)

    IF multiple_intents_detected(message):
        RETURN ask_client_to_prioritize(intents)    # правило з відкритого питання

    flow = get_dialog_flow(conversation.current_stage, intent.service_id)

    missing_fields = flow.required_fields - conversation.collected_context.keys()

    IF missing_fields.count > 0 AND clarifying_questions_asked < flow.max_clarifyin
        RETURN ask_for(missing_fields[0])

    card = get_knowledge_card(intent, conversation.collected_context)

    IF card.status == 'do_not_use':
        RETURN escalate_to_human(conversation, reason='outdated_card')

    IF matches_any(conversation.collected_context, card.handoff_condition):
        RETURN escalate_to_human(conversation, reason='handoff_condition_met')

    send(card.answer_short OR card.answer_full)
    IF card.disclaimer: send(card.disclaimer)
    IF card.next_question: ask(card.next_question)

    advance_stage(conversation, flow.next_stage_on_complete)

```

Важливе правило для кількох намірів одразу (це відповідає на прогалину з попереднього документа): якщо клієнт запитує про 2+ послуги в одному повідомленні — бот НЕ веде паралельно, а явно перепитує пріоритет:

“Бачу два питання — про карту побиту і про довіреність. З чого почнемо?”

5. Draft системного промпту для Claude API (Шар 3 — безпека)

Цей текст іде в system prompt при виклику `/v1/messages` з Orakul:

Ти — асистент продажу Kompas Migracji. Твоя задача — вести клієнта від першого питання до оплати й передачі справи спеціалісту, використовуючи ТІЛЬКИ дані з бази знань (`knowledge_cards`), передані тобі в контексті запиту.

СУВОРІ ПРАВИЛА:

1. Ніколи не гарантуй позитивне рішення у справі.
2. Ніколи не вигадуй строки, ціни чи назви установ — якщо їх немає в картці, скажи, що це уточнюється на консультації.
3. Якщо картка має статус `'do_not_use'` або `'needs_legal_review'` — не використовуй її дані як остаточні, познач це клієнту і передай людині.
4. Не показуй клієнту дві ціни з різних періодів як актуальну — бери лише картки зі статусом `'actual'`.
5. Не приймай рішення у складній справі (спадщина, неповнолітні, спори, судові оскарження) — одразу передавай людині за `handoff_rules`.
6. Не розкривай платіжні дані, доки не визначено послугу, суму й отримувача.
7. Якщо не впевнений — став уточнююче питання, а не вигадуй відповідь.

Формат відповіді: коротко, по суті, українською (або мовою клієнта), без зайвого канцеляриту.

6. Інтеграційні нотатки під Orakul (Next.js 14 + Supabase + Claude API)

- **Endpoint для бота:** одна точка входу `/api/bot/message`, яка отримує `conversation_id + text`, читає `collected_context` з Supabase, підтягує релевантні `knowledge_cards` (пошук за `trigger_phrases`, можна через `pgvector-embedding` пізніше), формує system prompt із блоку 5 + вставляє знайдені картки як контекст у

user-повідомлення.

- **Класифікація наміру:** на MVP-етапі можна класифікувати інтенст прямим викликом Claude (structured JSON output), не окремою ML-моделлю — простіше і швидше для старту.
 - **Ескалація до людини:** при `escalate_to_human` — статус `conversation` → `handed_off`, створюється задача у CRM (Мілена / Олександр / юрист), клієнту йде повідомлення на кшталт "Передаю ваш запит колезі, вона зв'яжеться найближчим часом".
 - **Оновлення актуальності карток:** окреме адмін-view (можна прямо в Supabase Studio на старті) для позначення `status` і `last_reviewed_at` — це має робити людина (Мілена/Олександр), не бот.
-

7. Що варто зробити першим (MVP-скоуп для Грека)

1. Розгорнути таблиці з розділу 2.
2. Засідувати 10–15 карток по найчастіших запитах (карта побиту, шлюб, довіреність) — можна взяти з реальних чатів Мілени.
3. Зробити просту версію стейт-машини з розділу 4 без embedding-пошуку (тільки keyword-matching по `trigger_phrases`).
4. Підключити system prompt із розділу 5.
5. Протестувати на 5–10 реальних діалогах з історії Мілени — порівняти відповіді бота з тим, що реально відповідала Мілена.